Interfata

- Interfata - contract, protocol de comunicare intre obiecte
- Clasa - implementeaza acel contract
- O interfata contine declaratii de metode abstracte, default, statice si date membru
- Orice data membru declarata in interfata este implicit public, static, final
- Toate metodele declarate intr-o interfata sunt implicit publice
- O interfata poate mosteni o alta interfata si poate adauga metode noi
- O interfata poate sa mosteneasca mai multe interfete, cuvant rezervat: **extends**

Genericitate

- Problema :
  - Construiti o structura de date: o stiva, o lista inlantuita, un vector, un graf, un arbore, etc.
  - Care este tipul de date pe care il vom folosi pentru reprezentarea elementelor?

```
public class Stack {
    private Object[] items;
    private int vf=0;

}

Stack stack = new Stack();
stack.push(100);
stack.push(new Rectangle());
stack.push("Hello World!");
String s = (String) stack.peek();
```

Cast!

```
public class Stack {
    private Object[] items;
    private int vf=0;
    public Stack(){…}
    public void push (Object item) {…}
    public Object peek() {…}
}

//testStack.java
Stack stack = new Stack();
stack.push(100);
stack.push(new Rectangle());
stack.push("Hello World!");

while (!stack.isEmpty())
{
        String s = (String)stack.pop();
        System.out.println(s);
}
```

Tipuri generice

- Permit parametrizarea tiprilor de date
- Asemanatoare cu template din C++

Conventii de numire a tipurilor

- E - Element
- K - Key
- N - Number
- T - Type
- V - Value

Exemple :

```
public class Stack<E> { … }
public class Node<T> { … }
public interface Pair<K, V> { … }
public class PairImpl<K, V> implements Pair<K, V> {...}
```

Instantierea tipurilor generice

```
public class Stack<E>{
    private E[] items;
    private int vf=0;
    public void push(E elem) {…}
    public E peek() {…}
}
```

- Cand se creeaza instante ale unor clase generice, nu se pot folosi tipurile primitive: int, byte, char, float, double,...
- Se folosesc clasele asociate:

Tip primitiv	Clasa învelitoare
boolean	Boolean
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double

Stack<int> si=new Stack<int>( ); //eroare la compilare
Stack<Integer> si = new Stack<Integer>( );

Autoboxing
public class Stack<E>{

}

Stack<Integer> si=new Stack<Integer>
si.push(23);
si.push(new Integer(23));
int val=si.peek( );

Tablouri cu elemente de tip generic

```
3   public class Stack<E> {
4       private E[] items=new E[20];
5
6
```

Type parameter 'E' cannot be instantiated directly

- Nu pot fi create folosind operatorul new
- Doar : E[ ] items = (E[ ])new Object[20]; //warning la compilare

Tablouri cu elemente de tip generic - alternative

- Se creeaza un tablou cu elemente de tip Object, iar cand se cere un element se face cast explicit:

```
public class Stack<E> {
    Object[] items=new Object[20];
    int vf=0;

    . . .
    public E peek() {
        if (vf>0)
            return (E)items[vf-1];
        else return null;
}
```

- Metodele generice definite in clase care nu declara tipuri generice

```
class Util {
    public static <T> int countNullValues(T[] anArray){
        int count = 0;
        for (T e : anArray)
            if (e == null) {
                ++count;
            }
        return count;
    }
}
```

```
//apel metoda generica
int k= Util.countNullValues(new String[]{"a", null, "b"});
int j= Util.countNullValues(new Integer[]{1, 2, null, 3, null});
```

- Tipul generic al unei metode generice poate sa fie diferit de tipul generic al clasei in care e delcarata metoda generica

```
public class Stack<E> {
    E[] items;
    int vf=0;
    public static <T> void copiaza(T[] elems, Stack<T> st) {
        for(T e:elems)
            st.push(e);
    }
}


Stack<Integer> st=new Stack<>(Integer.class);
st.push(2);
st.push(3);

Stack.copiaza(new Integer[]{4,5,6},st);
System.out.println(st.peek());
```

Putem avea atribute statice de tip generic?

```
public class Singleton <T> {
    public static T getInstance() {
        if (instance == null)
            instance = new Singleton<T>();

        return instance;
    }
    private static T instance = null;
}
```

```
class MobileDevice<T> {
    private  static T os;
    // ...
}
MobileDevice<Smartphone> phone = n
MobileDevice<Pager> pager = new Mo
MobileDevice<TabletPC> pc = new Mo
```

Variabila os este partajată de Smartphone, Pager şi TabletPC. Ea nu poate fi în acelaşi

```
Stack<String> t=new Stack<String>();
t.push("ana");
String s=t.peek();
```

```
//after Erasure
Stack t=new Stack();
t.push("ana");
String s=(String)t.peek();
```

Erasure

- In Java nu se creeaza o noua clasa pentru fiecare instanta a unei clase generice.
- La compilare, compilatorul "sterge" informatiile despre tipul gneric si inlocuieste fiecare variabila de tip generic cu limita superioara a tipului (de obicei Object) si unde este nevoie insereaza un cast explicit catre tipul generic.

```
Stack<String> t=new Stack<String>();
t.push("ana");
String s=t.peek();
```

```
//after Erasure
Stack t=new Stack();
t.push("ana");
String s=(String)t.peek();
```

Tipuri generice delimitate

- Se pot specifica constrangeri pentru tipul generic, folosind cuvantul **extends**.
  T **extends [C &] I1 [& I2 &...& In]** - T mosteneste clasa C si implementeaza interfetele I1,...In
- Cand se specifica constrangeri, la compilare, T este inlocuit cu primul element din expresia de constrangeri

  T **extends C //T este inlocuit cu C**

  T **extends C & I1 & I2 //T este inlocuit cu C**

  T **extends I1 & I2 //T este inlocuit cu I1**

  T **extends I1 //T este inlocuit cu I1**

```java
interface Comparable<E>{
     int compareTo(E element);
}
```

```java
public class ListaOrdonata<E extends Comparable<E>>
implements Lista<E> {
    private class Nod{
        E info;
        Nod urm;
        Nod(E elem){info=elem; urm=null;}
        Nod(E info, Nod urm){
            this.info=info;
            this.urm=urm;
        }
    }
    private Nod cap;
    private int count=0;
    public ListaOrdonata(){ cap=null;}
```

```java
public void adauga(E elem){
    count++;
    if (cap==null){
        cap=new Nod(elem);
        return;
    }
    if (elem.compareTo(cap.info)<0){
        cap=new Nod(elem,cap);
    }else {
        Nod p=cap;
        Nod r=p;
        while (p!=null && p.info.compareTo(elem)<0)
        {
            r=p;
            p=p.urm;
        }
 . . .
}
```

```java
public class Persoana implements Comparable<Persoana>{
    protected String nume;
    protected int varsta;
    public Persoana(){
        this("",0);
    }
    public Persoana(String nume, int varsta){
        this.nume=nume;
        this.varsta=varsta;
    }

    @Override
    public int compareTo(Persoana o) {
        return this.nume.compareTo(o.nume);
    }
 . . .
}
```

```java
public static void main(String[] args)
{

    Persoana p=new Persoana("Pop",12);
    Persoana p2=new Persoana("Ion",9);
    Persoana p3=new Persoana("Dan",14);
    ListaOrdonata<Persoana> l=new ListaOrdonata<Persoana>();
    l.adauga(p);
    l.adauga(p2);
    l.adauga(p3);
    for (int i=0; i<l.size(); i++)
        System.out.println(l.get(i));
}
```

Wildcards

- Wildcard-urile sunt utilizate atunci cand dorim sa folosim o structura de date generica si nu dorim sa limitam tipul de date din colectia respectiva

```java
public static void printCollection2(ListaOrdonata<?> c)
{
    for(int i=0; i<c.size();i++)
        System.out.println(c.get(i));

}

ListaOrdonata<Persoana> personList=new ListaOrdonata<Persoana>();
ListaOrdonata<Student> studList=new ListaOrdonata<Student>();
```

- Limitare : nu putem adauga elemente arbitrare intr-o colectie cu wildcard-uri:

```java
ListaOrdonata<?> c = new ListaOrdonata<String>();  // Operatie permisa
c.adauga(23);  // Eroare la compilare
```

Eroarea apare deoarece nu putem adauga intr-o colectie generica decat elemente de un anumit tip, iar wildcard-ul nu indica un tip anume.
De fapt, nu putem apela nicio metoda a clasei ListaOrdonata care contine in signatura ei un parametru de tip generic.

Wildcard-uri delimitate superior

- Daca se specifica o limita suoperioara pentru ? se pot apela metode apartinand clasei sau interfetei din limita superioara.

```java
public static void printCollection(ListaOrdonata<? extends Persoana> c)
{
    for(int i=0; i<c.size();i++)
        System.out.println(c.get(i));

}
```

```
ListaOrdonata<Student> studList=new ListaOrdonata<Student>();

Student s=new Student("x",12,3);
Student s1=new Student("a",14,1);
Student s3=new Student("z",9,2);
studList.adauga(s);
studList.adauga(s1);
studList.adauga(s3);

printCollection(studList);
```

Wildcard-uri delimitate inferior

```
public static void printCollection1(ListaOrdonata<? super Student> c)
{
    for(int i=0; i<c.size();i++)
        System.out.println(c.get(i));

}


Persoana p=new Persoana("Pop",12);
Persoana p2=new Persoana("Ion",9);
Persoana p3=new Persoana("Dan",14);

ListaOrdonata<Persoana> personList=new
ListaOrdonata<Persoana>();
printCollection1(personList);
```

Problema

```
public class Student extends Persoana {
    private static final int COD_UNIVERSITATE = 15;
    private int anStudiu;
}


ListaOrdonata<Student> studList=new ListaOrdonata<Student>();
```

Type parameter 'Generics.Student' is not within its bound; should implement 'java.lang.Comparable<Generics.Student>'

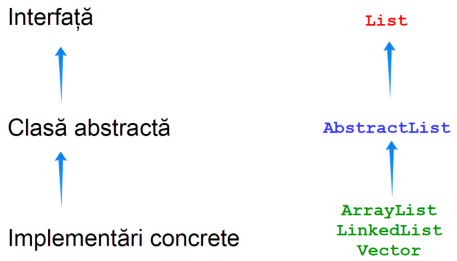
Soluția

```
public class ListaOrdonata<E extends Comparable<? super E>> implements Lista<E> {
```

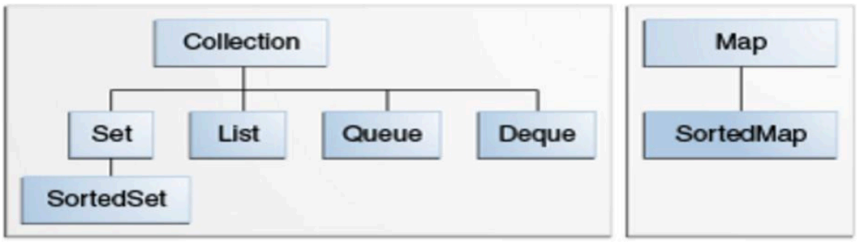
Cadrul colectiilor Java

- O colectie este un obiect care grupeaza mai multe elemente intr-o singura unitate (ex: vectori, liste inlantuite, stive, multimi, dictionare, etc)
- Reutilizarea codului
- Reducerea efortului de programare
- Cresterea vitezei si calitatii unei aplicatii
- Folosesc tipuri generice

Arhitectura colectiilor



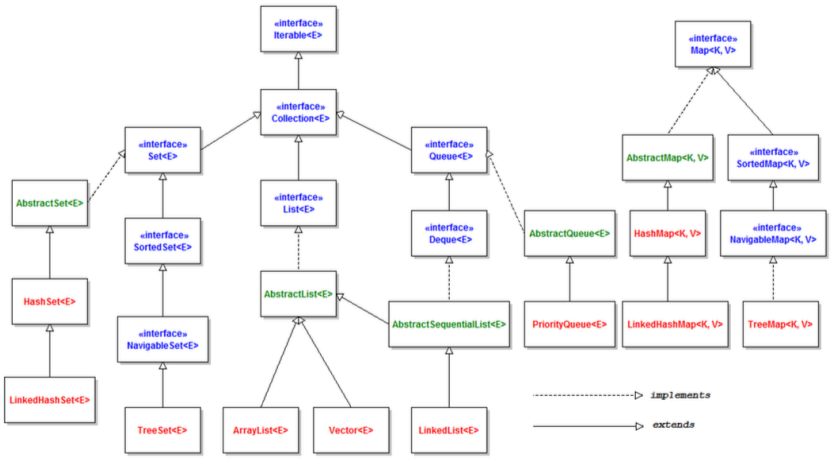
Interfete



Implementari - reprezentare

Interfața	Hash	Array	Tree	Linked	Hash+Linked
Set	HashSet		TreeSet		LinkedHashSet
List		ArrayList Vector		LinkedList	
Queue					
Deque		ArrayDeque		LinkedDeque	
Map	HashMap Hashtable		TreeMap		LinkedHashMap

Ierarhia colectiilor

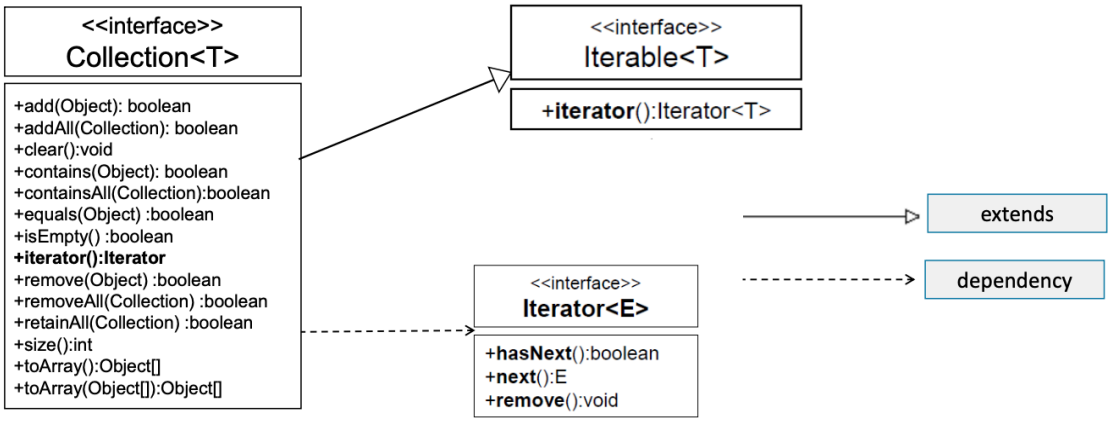


Class diagram of Java Collections framework

Declarare si instantiere obiecte

Set set = new HashSet( ); *//raw generic type Object*
ArrayList<Integer> list = new ArrayList<>( );
List<Integer> list = new ArrayList<>( );
List<Integer> list = new LinkedList<>( );
List<Integer> list = new Vector<>( );
Map<Integer, String> map = new HashMap<>( );

Interfata Collection



Parcurgerea colectiilor

- **for-each**

```
public void list(Collection<T> items) {
```

```
for (Item item : items) {
  System.out.println(item);
}
```

=

- **Cu iterator**

```
public void list(Collection<T> items) {
```

```
Iterator<Item> it = items.iterator();
while(it.hasNext()) {
  Item item = it.next();
  System.out.println(item);
}
```

Clasa **Collections** contine exclusiv metode statice pentru sortarea unei colectii, pentru determinarea minimului, maximului, inversarea unei colectii si cautarea unei valori.

Interfata List

- O lista este o colectie **ordonata**. Listele pot contine elemente **duplicate**.
- Pe langa operatiile mostenite de la Collection, interfata List defineste urmatoarele operatii :
  T get(int index) - intoarce elementul de la pozitia index
  T set(int index, T element) - modifica elementul de la pozitia index
  void add(int index, T element) - adauga un element la pozitia index
  T remove(int index) - sterge elementul de la pozitia index
- List poseda doua implementari standard :
  ArrayList - implementare sub forma de vector. Accesul la elemente se face in timp constant, O(1)
  LinkedList - implementare sub forma de lista dublu inlantuita. Prin urmare, accesul la un element nu se face in timp constant, fiind necesara o parcurgere a listei O(n)
- Printre algoritmii implementati se numara :
  sort - realizeaza sortarea unei liste
  binarySearch - realizeaza o cautare binara a unei valori intr-o lista

Interfata Set

- Un Set (multime) este o colectie ce nu poate contine elemente duplicate.
- Interfata Set contine doar metodele mostenite din Collection, la care se adauga restrictii astfel incat elementele duplicate sa nu poata fi adaugate.
- Avem trei implementari utile pentru Set :
  HashSet: memoreaza elementele sale intr-o tabela de dispersie (hash table); este implementarea cea mai performanta, insa nu avem garantii asupra ordinii de parcurgere. Doi iteratori diferiti pot parcurge elementele multimii in ordine diferita.
  TreeSet: memoreaza elementele sale sub forma de arbore rosu-negru; elementele sunt ordonate pe baza valorilor sale. Implementarea este mai lenta decat HashSet.
  LinkedHashSet: este implementata ca o tabela de dispersie. Diferenta de HashSet este ca LinkedHashSet mentine o lista dublu-inlantuita peste toate elementele sale. Prin urmare elementele raman in ordinea in care au fost inserate. O parcurgere a LinkedHashSet va gasi elementele mereu in aceasta ordine.

Interfata Map

- Un Map este un obiect care mapeaza chei la valori. Intr-o astfel de structura nu pot exista chei duplicate.
- Fiecare cheie este mapata la exact o valoare.
- Map reprezinta o modelare a conceptului de functie : primeste o entitate ca parametru (cheia), si intoarce o alta entitate (valoarea).
- Cele trei implementari pentru Map sunt:
  HashMap
  TreeMap
  LinkedHashMap
- Particularitatile de implementare corespund celor de la Set

Compararea elementelor

Pentru compararea a doua elemente dintr-o colectie avem doua posibilitati:

1) Entitatile colectiei implementeaza interfata java.lang.Comparable<T>

```
public interface Comparable<T>{
    int compareTo(T o);
}
```

2) Defenim un comparator care implementeaza java.util.Comparator<T>

```
public interface Comparator<T>{
    int compare(T o1, T o2);
}
```